

Sistemi Operativi (Modulo II)

Capitolo 6 — Gestione I/O e Memoria Secondaria

Luca Argentino

A.A. 2025–2026

Indice

6	Gestione I/O e Memoria Secondaria	2
6.1	Interfacce I/O	2
6.2	Classificazione dei sistemi di I/O	3
6.3	Dispositivi a blocchi e a caratteri	4
6.3.1	Dispositivi a blocchi	4
6.3.2	Dispositivi a caratteri	4
6.4	Progettazione del sistema di I/O	5
6.4.1	Buffering	5
6.4.2	Caching	5
6.4.3	Spooling	6
6.5	Archiviazione di rete: NAS e SAN	7
6.5.1	NAS — Network-Attached Storage	7
6.5.2	SAN — Storage Area Network	8
6.6	Memoria secondaria: dischi rotazionali e SSD	11
6.6.1	Solid State Drive (SSD)	11
6.6.2	Dischi rotazionali (HDD)	12
6.7	Disk Scheduling	13
6.7.1	FCFS — First Come, First Served	14
6.7.2	SSTF — Shortest Seek Time First	15
6.7.3	LOOK — Algoritmo dell'Ascensore	16
6.7.4	C-LOOK — Circular LOOK	17
6.7.5	Problema comune a LOOK e C-LOOK: starvation parziale	18
6.8	RAID	18
6.8.1	Motivazione e concetto	18
6.8.2	RAID 0 — Striping	19
6.8.3	RAID 1 — Mirroring	19
6.8.4	RAID 4 — Parità su disco dedicato (Teorico)	20
6.8.5	RAID 5 — Parità distribuita	23
6.8.6	RAID 6 — Doppia parità	23
6.8.7	Riepilogo dei livelli RAID	24
6.9	Considerazioni finali	24

Capitolo 6

Gestione I/O e Memoria Secondaria

Questo capitolo tratta due macro-argomenti strettamente legati tra loro: la **gestione dell'Input/Output** da parte del sistema operativo e la **memoria secondaria** (dischi rotazionali, SSD, RAID). Capire come il kernel interagisce con i dispositivi periferici è fondamentale per comprendere le scelte di progettazione che rendono un sistema operativo efficiente e affidabile.

6.1 Interfacce I/O

Il sistema operativo offre un'**astrazione uniforme** per gestire dispositivi di natura completamente diversa (dischi SCSI, tastiere, mouse, periferiche ATAPI). Come illustrato nello schema, questa modularità è ottenuta tramite una rigida architettura a livelli che separa il software dall'hardware:

- **Kernel e Driver (Livello Software - Riquadri blu):** Il sistema operativo non comunica mai direttamente con l'elettronica. Delega la comunicazione a moduli software specifici chiamati *device driver* (es. SCSI driver, keyboard driver), i quali conoscono lo specifico "linguaggio" necessario per gestire quella famiglia di dispositivi.
- **Controller (Livello Hardware Intermedio - Riquadri gialli):** Ogni driver comunica con il rispettivo *controller* hardware. Il controller è il chip (spesso integrato sulla scheda madre o in una scheda di espansione) che riceve i comandi digitali dal driver e li traduce in segnali elettrici.
- **Dispositivi (Livello Hardware Finale - Riquadri gialli):** Rappresenta la periferica fisica vera e propria (la tastiera meccanica, il sensore ottico del mouse, il lettore floppy) che esegue materialmente l'operazione di I/O o l'interazione con l'utente.

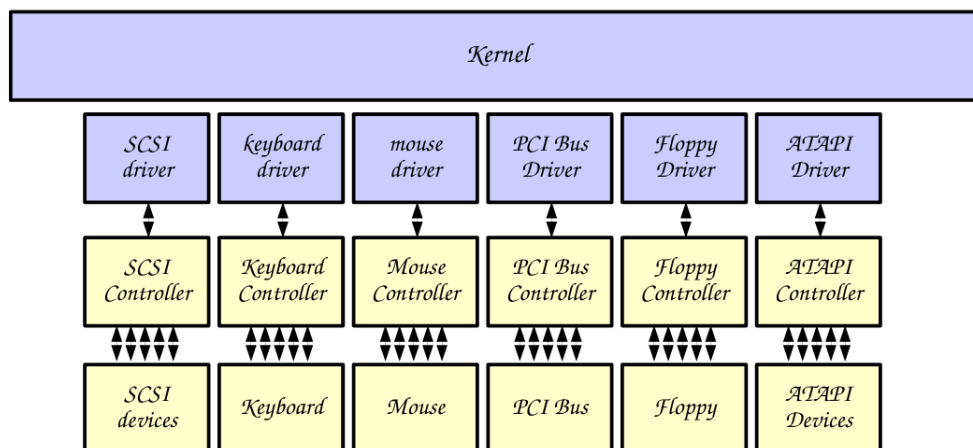


Figura 6.1: Architettura a livelli dell'I/O: la comunicazione fluisce dal Kernel ai Driver software, per poi attraversare il confine hardware verso i Controller e infine ai dispositivi fisici.

Nota – Separazione tra Controller e Dispositivo

Lo schema evidenzia una distinzione fondamentale spesso trascurata: il controller e il dispositivo non sono la stessa cosa. Ad esempio, il *Keyboard Controller* è l'interfaccia elettronica interna al computer che decodifica i segnali, mentre la *Keyboard* in basso è l'oggetto fisico esterno dotato di tasti.

6.2 Classificazione dei sistemi di I/O

I dispositivi di I/O sono estremamente eterogenei. Il sistema operativo li classifica secondo diversi **assi di variazione**, descritti nella Tabella 6.1.

Tabella 6.1: Assi di classificazione dei dispositivi I/O.

Aspetto	Variazioni possibili
Modalità di trasferimento	Caratteri / Blocchi
Modalità di accesso	Sequenziale / Random
Trasferimento	Sincrono / Asincrono
Condivisione	Dedicato / Condivisibile
Velocità	Da pochi byte/s fino a Gigabyte/s
Direzione di I/O	Sola lettura / Sola scrittura / Lettura-scrittura

Questa classificazione non è solo teorica: influenza concretamente il modo in cui il driver e il kernel gestiscono il dispositivo. Ad esempio, un dispositivo *sequenziale* non può essere soggetto a *seek* (salto a posizione arbitraria), mentre un dispositivo *dedicato* non può essere condiviso da più processi contemporaneamente senza serializzazione.

6.3 Dispositivi a blocchi e a caratteri

La distinzione più importante per la memoria secondaria è quella tra dispositivi a **blocchi** e dispositivi a **caratteri**.

6.3.1 Dispositivi a blocchi

Definizione – Dispositivo a blocchi

Un **dispositivo a blocchi** trasferisce dati in unità di dimensione fissa, tipicamente da 512 a 4096 byte (un *blocco* o *settore*). Il driver espone operazioni di **read**, **write** e **seek** sulle posizioni di blocco.

Esistono tre modalità di accesso ai dispositivi a blocchi:

1. **Raw I/O** — il processo accede direttamente ai blocchi fisici, bypassando il file system. Utile per database e applicazioni ad alte prestazioni che gestiscono da sé la cache.
2. **Accesso tramite file system** — lo strato del file system traduce operazioni su *file* (con nome, offset in byte, ...) in operazioni su blocchi fisici. Il driver fornisce solo l'accesso raw; il file system è uno “strato superiore”.
3. **Memory-mapped I/O** — il contenuto di un file viene *mappato* in un intervallo dello spazio di indirizzamento virtuale del processo. L'accesso avviene tramite normali istruzioni **load/store** della CPU, e il kernel gestisce le page fault per portare i dati in RAM.

Nota – Memory-mapped I/O e demand paging

La memoria mappata su file sfrutta esattamente lo stesso meccanismo di *demand paging* studiato nel Capitolo 5: le pagine vengono caricate su richiesta (page fault) e il backing store non è la swap, bensì il file stesso. Questo permette di condividere librerie (es. `libc.so`) tra processi senza duplicarle in RAM.

6.3.2 Dispositivi a caratteri

Definizione – Dispositivo a caratteri

Un **dispositivo a caratteri** trasferisce dati un byte (o un carattere) alla volta. Il driver espone operazioni di **get** e **put** di singoli caratteri, oppure supporta una modalità *bufferizzata* che lavora riga per riga.

Esempi tipici di dispositivi a caratteri: tastiera, mouse, porte seriali, modem. La lettura riga per riga (*line discipline*) è gestita dal kernel e permette l'editing locale (backspace, Ctrl-C, ...) prima che i dati raggiungano il processo.

6.4 Progettazione del sistema di I/O

Il kernel utilizza quattro tecniche principali per ottimizzare le prestazioni del sottosistema I/O: **buffering**, **caching**, **spooling** e **I/O scheduling**. Le prime tre vengono trattate nelle sottosezioni seguenti; l'I/O scheduling è approfondito nella sezione dedicata al disk scheduling (Sezione 6.7).

6.4.1 Buffering

Definizione – Buffer

Un **buffer** è un'area di memoria temporanea in cui i dati vengono trattiene *prima* di essere trasferiti alla destinazione finale. Nel buffer risiede l'**unica istanza** di quell'informazione in quel momento.

Il buffering nasce da **tre motivazioni distinte**:

1. **Differenza di velocità tra produttore e consumatore.** Il produttore (es. un processo che scrive su disco) potrebbe essere molto più veloce del consumatore (il disco fisico), o viceversa. Il buffer “assorbe” le fluttuazioni di velocità, evitando che il produttore rimanga bloccato ad aspettare il consumatore lento.

Esempio – Scrittura su disco

Un processo scrive dati a 1 GB/s (velocità RAM), ma il disco scrive a 100 MB/s. Senza buffer, il processo aspetterebbe il disco per ogni scrittura. Con un buffer in RAM, il processo scrive nel buffer (veloce) e il kernel svuota il buffer nel disco in background.

2. **Differenza nelle dimensioni delle unità di trasferimento.** Il produttore potrebbe produrre dati in unità diverse rispetto a quelle richieste dal dispositivo. Ad esempio, un processo scrive un byte alla volta, ma il disco richiede blocchi da 4 KB. Il buffer accumula i byte finché non si raggiunge la dimensione minima per il trasferimento.
3. **Semantica di copia delle operazioni di I/O.** Quando un processo invoca `write()`, si aspetta che il dato scritto non venga modificato prima che la scrittura sia completata. La *copy semantics* garantisce che il kernel copi il dato dal buffer dell'utente nel proprio buffer kernel, così il processo può modificare i propri dati subito dopo il ritorno dalla `write()`.

6.4.2 Caching

Definizione – Cache

Una **cache** mantiene in memoria primaria una *copia* di informazioni che risiedono in memoria secondaria. A differenza del buffer, il dato originale esiste anche nel suo supporto definitivo (il disco): la cache ne è solo una replica più velocemente accessibile.

La distinzione concettuale tra buffer e cache è fondamentale:

- Nel **buffer**: l'informazione esiste *solo* nel buffer (fino a che non viene scritta definitivamente).
- Nella **cache**: l'informazione esiste *sia* nella cache *sia* nel supporto originale (disco).

Nota – Page cache in Linux

Linux utilizza la *page cache* come cache per i blocchi del file system. Ogni accesso a un file viene prima cercato in page cache; solo in caso di miss il blocco viene letto dal disco e inserito in cache. Le pagine modificate (*dirty pages*) vengono riscritte sul disco in modo asincrono dal kernel thread `pdflush/writeback`.

6.4.3 Spooling

Definizione – Spool

Uno **spool** è un buffer particolare utilizzato per gestire dispositivi che non possono accettare flussi di dati distinti (da processi diversi) contemporaneamente. I dati vengono accodati nello spool e inviati al dispositivo in modo serializzato.

L'esempio classico è la **stampante**: più processi vogliono stampare contemporaneamente, ma la stampante deve ricevere un documento alla volta (altrimenti le pagine di documenti diversi si mescolerebbero). Il *print spooler* (es. `CUPS` in Linux) riceve tutti i job di stampa, li coda nello spool e li invia alla stampante uno alla volta.

6.5 Archiviazione di rete: NAS e SAN

La memoria secondaria non deve necessariamente essere locale alla macchina. Esistono due approcci architetturali per la condivisione della memoria di massa in rete.

6.5.1 NAS — Network-Attached Storage

Definizione – NAS

Un **Network-Attached Storage** (NAS) è un dispositivo (o cluster di dispositivi) di memoria secondaria connesso direttamente alla rete locale, accessibile dai client tramite protocolli di file system di rete (NFS, SMB/CIFS, ...).

- **Unità che consentono mount remoto**
- **Memoria secondaria condivisa**

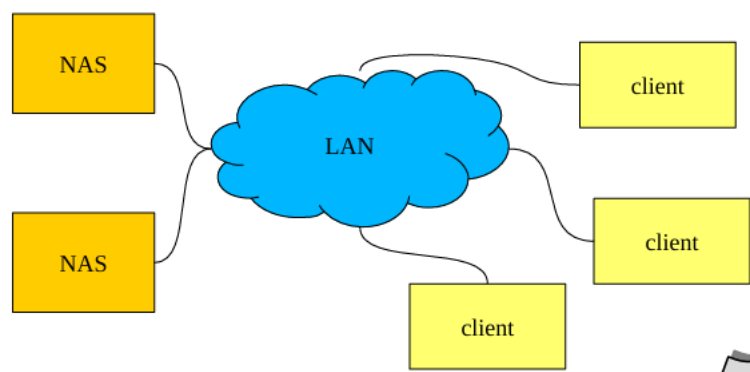


Figura 6.2: Architettura NAS: memoria secondaria condivisa tramite LAN.

Come funziona il mount remoto

Nei sistemi Unix/Linux, ogni file system — locale o remoto — viene reso accessibile tramite l'operazione di **mount**: il file system viene “agganciato” a una directory esistente nell'albero locale (il *mount point*). Da quel momento in poi, tutto ciò che si trova sotto quella directory è in realtà sul dispositivo montato.

Con un NAS il meccanismo è identico, ma il dispositivo non è un disco locale: è un server remoto raggiungibile tramite rete. Il sistema operativo client installa un **driver di file system di rete** (es. il driver NFS su Linux) che intercetta tutte le operazioni su file (*open*, *read*, *write*, *stat*, ...) dirette al mount point e le **traduce in richieste di rete** verso il server NAS.

Accesso concorrente e coerenza

Poiché più client possono montare lo stesso NAS contemporaneamente, nasce immediatamente un problema di **coerenza**: cosa succede se due client leggono e scrivono lo stesso file nello stesso momento?

I protocolli di file system di rete gestiscono questo con meccanismi di **locking** e **semantica di coerenza**:

- **NFS (Network File System)**: sviluppato da Sun Microsystems, è lo standard su sistemi Unix/Linux. NFS è tradizionalmente *stateless* (il server non ricorda lo stato dei client): ogni richiesta è autosufficiente. Questo semplifica il recovery in caso di crash del server, ma rende il locking più complesso (gestito separatamente dal protocollo NLM — Network Lock Manager).
- **SMB/CIFS (Server Message Block / Common Internet File System)**: è il protocollo usato da Windows (e da Samba su Linux). È *stateful*: il server mantiene lo stato delle sessioni aperte, facilitando il locking ma complicando il recovery.

Confronto NAS vs. disco locale

Tabella 6.2: Confronto tra accesso a disco locale e accesso tramite NAS.

Aspetto	Disco locale	NAS
Velocità	Alta (SATA/NVMe, μs -ms)	Limitata dalla rete (LAN: ~ 1 Gb/s, latenza ms)
Condivisione	Solo il sistema locale	Più client contemporaneamente
Trasparenza	Completa	Completa (stessa API POSIX)
Fault tolerance	Dipende dall'HW locale	Il NAS può avere RAID interno
Gestione	Locale	Centralizzata sul NAS
Costo per GB	Basso	Più alto (hw di rete + server)

6.5.2 SAN — Storage Area Network

Definizione – SAN

Una **Storage Area Network** (SAN) è una rete ad alta velocità dedicata (separata dalla LAN) che connette server a array di storage (che altro non sono che batterie di dischi). I server vedono lo storage SAN come se fosse un disco locale.

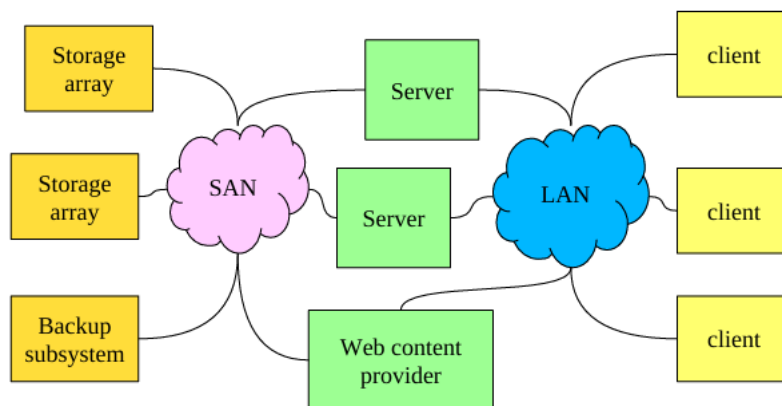


Figura 6.3: Architettura SAN: rete dedicata per lo storage ad alta velocità.

La differenza fondamentale tra NAS e SAN:

- **NAS** opera a livello di *file system* (il protocollo trasferisce file e directory).
- **SAN** opera a livello di *blocchi* (il protocollo trasferisce blocchi grezzi, come se fossero dischi locali). Il file system è gestito dal server, non dallo storage.

Il modello concettuale: separare il calcolo dallo storage

L'idea fondamentale di una SAN è **disaccoppiare** la macchina che esegue i processi (il *server*) dalla macchina che conserva i dati (lo *storage array*). Prima delle SAN, i dischi erano fisicamente dentro il server: se il server si guastava, i dati erano inaccessibili. Con una SAN, i due componenti vivono su macchine separate collegate da una rete dedicata.

Dal punto di vista del server, tuttavia, non cambia nulla: il sistema operativo esegue le normali operazioni di mount e vede la partizione dello storage array come un qualsiasi disco locale. Tutta la complessità della rete SAN è gestita dallo strato hardware/firmware del controller SAN, in modo completamente trasparente al kernel.

Esempio – I laboratori Ercolani — Università di Bologna

I laboratori informatici del dipartimento DISI in via Ercolani sono un esempio concreto di architettura SAN. L'infrastruttura funziona così:

1. Gli **storage array** (batterie di dischi con RAID interno) sono macchine fisiche dedicate esclusivamente alla conservazione dei dati: home directory degli utenti, immagini del sistema operativo, dati dei corsi, ...
2. I **server** (macchine che eseguono i sistemi operativi e i servizi) sono connessi agli storage array tramite la SAN. All'avvio, ogni server esegue il mount delle partizioni di cui ha bisogno: dal suo punto di vista è *come se avesse un disco locale*, ma quel disco è fisicamente su un'altra macchina.
3. I **client** (le postazioni nei laboratori) si connettono ai server tramite la normale LAN: ricevono il desktop, eseguono applicazioni, accedono ai propri file. Non sanno nulla della SAN sottostante.

Il risultato è un **sistema distribuito** in cui la gestione della memoria secondaria è centralizzata e separata dal calcolo.

Vantaggi principali di una SAN

1. Alta disponibilità: il server può andare giù senza perdere i dati. Poiché i dati risiedono sullo storage array e non sul server, un guasto del server **non compromette i dati**. È sufficiente:

1. Avviare un server di riserva (o riparare quello guasto),
2. Rimontare le stesse partizioni dello storage array,
3. Riprendere il servizio.

I dati non si sono mai mossi: erano sullo storage array durante il guasto e ci sono ancora dopo. Questo è impossibile con l'architettura tradizionale (disco nel server): se il server va giù, i dati sono fisicamente inaccessibili finché non viene riparato.

Nota – Failover automatico

Negli ambienti enterprise, questo processo è automatizzato: un secondo server di *standby* monitora il server primario e, se rileva un guasto, monta automaticamente le stesse partizioni SAN e subentra nel giro di secondi. Questa tecnica si chiama **failover** e garantisce alta disponibilità (*high availability*, HA) senza intervento umano.

2. Condivisione dello storage tra server. Più server possono montare **partizioni diverse** dello stesso storage array. Questo porta a due vantaggi concreti:

- **Utilizzo efficiente della capacità:** invece di avere n server ognuno con il proprio disco parzialmente vuoto, la capacità totale è un unico pool condiviso, allocabile dinamicamente a chi ne ha bisogno.
- **Gestione centralizzata:** backup, monitoraggio, espansione della capacità vengono fatti in un unico posto (lo storage array), non su ogni singolo server. Negli Ercolani, ad esempio, il backup delle home directory riguarda lo storage array, non le singole postazioni.

Confronto NAS vs. SAN

La distinzione fondamentale tra NAS e SAN riguarda il **livello dell'astrazione** offerta ai client:

Tabella 6.3: Confronto NAS vs. SAN.

Aspetto	NAS	SAN
Astrazione offerta	File system (file e directory)	Blocchi grezzi (come un disco locale)
Protocollo	NFS, SMB/CIFS (su TCP/IP)	Fibre Channel, iSCSI
Chi gestisce il FS	Il NAS stesso	Il server che monta la partizione
Accesso concorrente	Sì (più client, stessi file)	No (tipicamente un server per partizione)
Latenza	Più alta (overhead protocollo)	Molto bassa (vicina al disco locale)
Complessità e costo	Bassa/media	Alta (rete dedicata, HW specializzato)
Caso d'uso tipico	Condivisione file tra utenti	Infrastrutture server, virtualizzazione

In sintesi: il NAS è pensato per *condividere file tra persone*; la SAN è pensata per *dare dischi veloci e affidabili ai server*.

6.6 Memoria secondaria: dischi rotazionali e SSD

Le due tecnologie dominanti per la memoria secondaria sono i **dischi rotazionali** (HDD) e i **Solid State Drive** (SSD).

6.6.1 Solid State Drive (SSD)

Definizione – SSD

Un **Solid State Drive** (SSD) è un dispositivo di memoria di massa basato su celle di memoria flash NAND, senza parti meccaniche in movimento.

Le caratteristiche principali degli SSD rispetto agli HDD tradizionali sono:

- **Nessuna fragilità meccanica:** non ci sono testine, bracci, piatti in rotazione. Resistono meglio a urti e vibrazioni.
- **Minor consumo energetico:** assenza di motore rotazionale. Fondamentale per laptop e dispositivi mobili.
- **Numero massimo di cicli di scrittura:** le celle flash si degradano dopo un certo numero di cicli di cancellazione/scrittura (tipicamente 10^3 – 10^5 cicli per cella). Per questo motivo, il firmware degli SSD include un *wear leveling* algorithm che distribuisce uniformemente le scritture su tutte le celle.
- **Velocità lettura > velocità scrittura:** la lettura di blocchi è molto più veloce della scrittura, che richiede un'operazione di cancellazione preventiva.
- **Si legge a blocchi, si scrive a “banchi”:** a differenza degli HDD, la cancellazione (necessaria prima di ogni scrittura) avviene su unità molto più grandi dei blocchi di lettura (*erase block*, tipicamente 128–512 blocchi da 4 KB = 512 KB – 2 MB). Questo introduce il problema della *write amplification*.
- **Accesso uniforme:** non esistendo meccanismi di seek, il tempo di accesso è praticamente identico per qualsiasi posizione del disco. Non vale più l'ottimizzazione del disk scheduling basata sulla *localizzazione* delle richieste.

Nota – Write Amplification negli SSD

Il **write amplification** si verifica perché per scrivere pochi byte su una cella, l'SSD deve: leggere l'intero erase block in RAM, modificare i byte voluti, cancellare l'intero erase block sul flash, riscrivere l'intero block. Il numero reale di byte scritti sul flash è molto maggiore di quello richiesto dall'applicazione. Il firmware minimizza questo effetto con algoritmi come il *garbage collection* e il *wear leveling*.

6.6.2 Dischi rotazionali (HDD)

Definizione – Disco rotazionale

Un **Hard Disk Drive** (HDD) è un dispositivo di memoria di massa basato su piatti magnetici in rotazione. La lettura/scrittura avviene tramite testine magnetiche mobili.

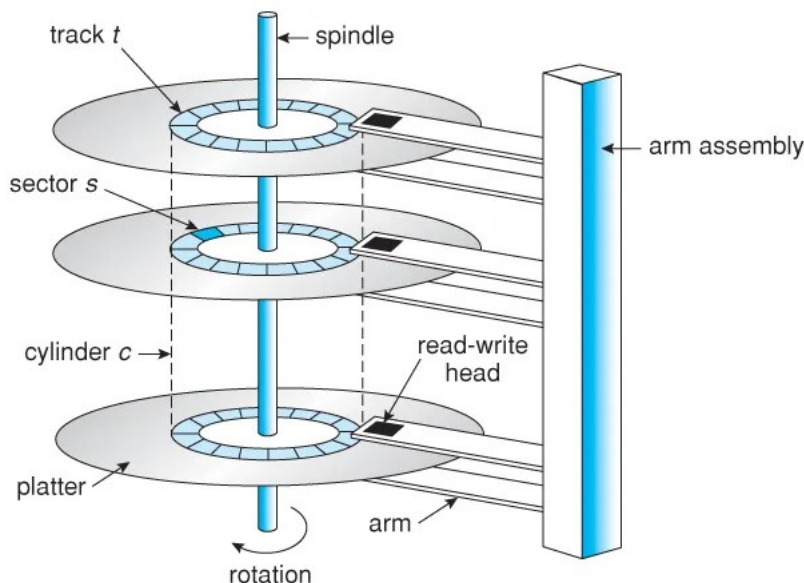


Figura 6.4: Struttura di un disco rotazionale: piatti, tracce e settori.

Un disco è organizzato gerarchicamente:

- **Piatti** (*platters*): dischi circolari magnetizzabili, uno o più per ogni HDD.
- **Tracce** (*tracks*): anelli concentrici su ogni piatto.
- **Settori** (*sectors*): suddivisioni di ogni traccia, tipicamente da 512 byte o 4 KB.
- **Cilindri** (*cylinders*): l'insieme delle tracce alla stessa distanza radiale su tutti i piatti.

I tre parametri fondamentali di un HDD sono:

- r — velocità di rotazione (rpm, *revolutions per minute*); valori tipici: 5400, 7200, 10000 rpm.
- T_s — tempo di seek medio (ms), ovvero il tempo medio necessario affinché la testina si sposti sulla traccia desiderata; valori tipici: 8–10 ms.
- V_r — velocità di trasferimento (byte/s).

Tempo di accesso

Definizione – Tempo di accesso al disco

Il **tempo di accesso** a un settore è la somma di tre componenti:

$$T_{\text{accesso}} = T_s + T_{\text{rot}} + T_{\text{transfer}}$$

dove:

- $T_s = \text{seek time}$: tempo per spostare la testina sulla traccia desiderata.
- $T_{\text{rot}} = \frac{1}{2r} = \text{ritardo rotazionale}$: tempo medio perché il settore desiderato arrivi sotto la testina (in media metà rotazione).
- $T_{\text{transfer}} = \frac{b}{V_r} = \text{tempo di trasferimento}$: dipende dalla quantità b di dati da leggere (assumendo che siano contigui sulla stessa traccia).

Esempio – Calcolo del tempo di accesso

Sia un disco con:

- $r = 7200 \text{ rpm} \Rightarrow T_{\text{rot}} = \frac{1}{2 \cdot 7200/60} \approx 4.2 \text{ ms}$
- $T_s = 9 \text{ ms}$
- $V_r = 100 \text{ MB/s}, b = 4 \text{ KB} \Rightarrow T_{\text{transfer}} \approx 0.04 \text{ ms}$

Il tempo totale di accesso è circa $9 + 4.2 + 0.04 \approx \mathbf{13.24 \text{ ms}}$. Si noti che il trasferimento è quasi trascurabile; il collo di bottiglia è il *seek* e il ritardo rotazionale. Questo motiva l'importanza del **disk scheduling**.

6.7 Disk Scheduling

Quando il sistema operativo riceve più richieste di I/O su disco, il **disk scheduler** decide l'ordine in cui soddisfarle. L'obiettivo è minimizzare il tempo di seek complessivo, il quale domina il tempo di accesso (come visto nell'esempio precedente).

Il gestore del disco può ricevere richieste da *molti processi concorrenti*; scegliere l'ordine ottimale di evasione può ridurre drasticamente il numero totale di seek.

Nei seguenti esempi usiamo sempre la stessa configurazione:

- Coda delle richieste (in ordine di arrivo): **98, 184, 39, 121, 14, 125, 65, 68**
- Posizione iniziale della testina: **54**
- Disco: tracce da 0 a 200

Nel mondo reale ovviamente mentre risolviamo richieste ne arrivano altre.

6.7.1 FCFS — First Come, First Served

Definizione – FCFS

La politica **First Come, First Served** (FCFS, o FIFO) serve le richieste nell'ordine in cui arrivano, senza alcuna ottimizzazione sulla posizione.

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54

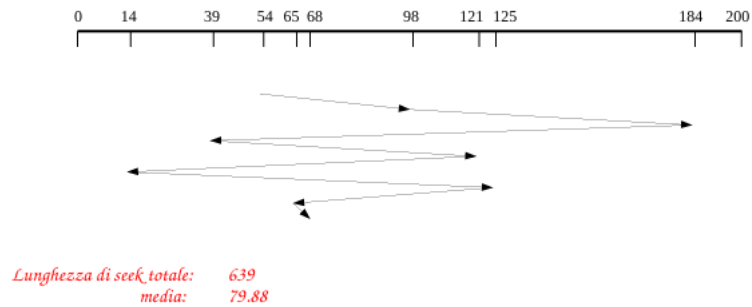


Figura 6.5: Esempio FCFS: spostamenti della testina.

Sequenza: 54 → 98 → 184 → 39 → 121 → 14 → 125 → 65 → 68

$$\text{Seek} = |98-54| + |184-98| + |39-184| + |121-39| + |14-121| + |125-14| + |65-125| + |68-65| = \mathbf{639}$$

Pro: **Fair** (nessun processo viene penalizzato). **No starvation** (ogni richiesta viene prima o poi servita).

Contro: Non minimizza il seek; la testina può fare percorsi molto inefficienti (avanti e indietro ripetutamente).

6.7.2 SSTF — Shortest Seek Time First

Definizione – SSTF

La politica **Shortest Seek Time First** (SSTF) serve sempre la richiesta che richiede il minor spostamento della testina dalla posizione corrente. In caso di parità, la direzione viene scelta casualmente.

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54

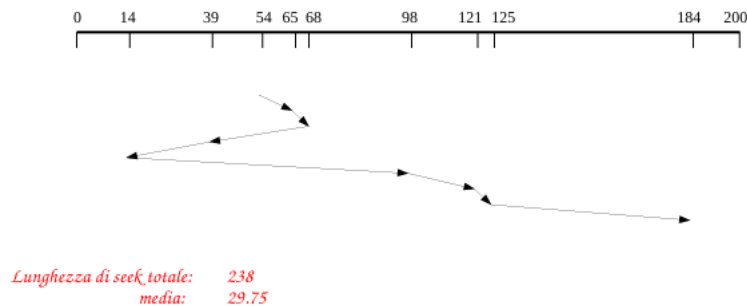


Figura 6.6: Esempio SSTF: spostamenti della testina.

Sequenza: 54 → 65 → 68 → 39 → 14 → 98 → 121 → 125 → 184

Seek totale = **238** (media: 29.75)

Pro: Molto più efficiente di FCFS in termini di seek totale.

Contro: Può causare **starvation**: se arrivano continuamente richieste vicine alla posizione corrente, le richieste “lontane” non vengono mai servite. Simile al problema dello *shortest job first* in CPU scheduling.

Nota – SSTF e starvation

Si immagini una testina ferma sul cilindro 50. Se arrivano di continuo richieste sui cilindri 45–55, le richieste sul cilindro 180 non verranno mai servite. Questo è esattamente il fenomeno di starvation: una richiesta potrebbe aspettare indefinitamente.

6.7.3 LOOK — Algoritmo dell'Ascensore

Definizione – LOOK

L'algoritmo **LOOK** (detto anche *algoritmo dell'ascensore*) gestisce la testina come un ascensore: la testina si sposta in una direzione, servendo tutte le richieste in quella direzione, e quando non ci sono più richieste in quella direzione, *inverte* la direzione e torna indietro.

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54

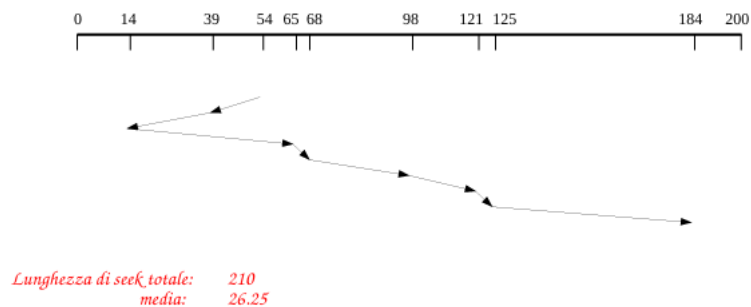


Figura 6.7: Esempio LOOK: la testina si muove come un ascensore.

Sequenza

(partendo in direzione discendente): 54 → 39 → 14 → 65 → 68 → 98 → 121 → 125 → 184

(partendo in direzione crescente): 54 → 65 → 68 → 98 → 121 → 125 → 184 → 39 → 14

Seek totale = **210** (media: 26.25)

Pro: Efficiente. Praticamente immune da starvation (ogni richiesta viene servita entro al massimo due passate complete).

Contro: Il tempo medio di attesa *non è omogeneo*: le tracce centrali del disco vengono visitate più frequentemente (la testina le attraversa in entrambe le direzioni), mentre le tracce agli estremi vengono visitate meno spesso. Le tracce centrali sono quindi **privilegiate** rispetto a quelle periferiche.

Nota – SCAN vs LOOK

La variante **SCAN** è simile a LOOK, ma invece di invertire quando raggiunge l'ultima richiesta, raggiunge sempre il bordo fisico del disco (traccia 0 o traccia massima) prima di invertire. LOOK è generalmente preferito perché evita seek inutili verso i bordi.

6.7.4 C-LOOK — Circular LOOK

Definizione – C-LOOK

C-LOOK ha lo stesso principio di LOOK, ma la scansione avviene in *una sola direzione*. Quando si raggiunge l'ultima richiesta nella direzione scelta, la testina **salta direttamente alla prima richiesta** (nella direzione opposta) senza servire le richieste intermedie durante il ritorno.

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54

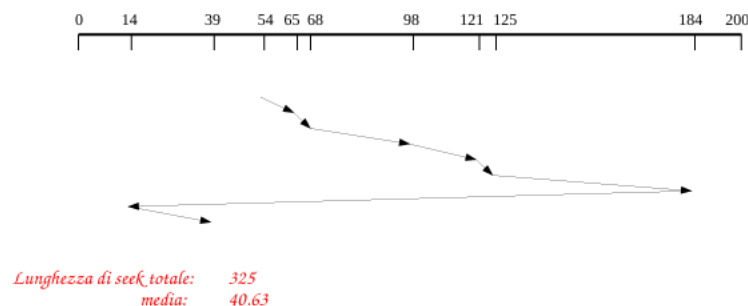


Figura 6.8: Esempio C-LOOK: scansione circolare in una sola direzione.

Sequenza: 54 → 65 → 68 → 98 → 121 → 125 → 184 → [salto] → 14 → 39

Seek totale = **325** (media: 40.63)

Pro: Tempo di attesa **più uniforme** rispetto a LOOK: tutte le tracce vengono servite con frequenza simile, eliminando il privilegio delle tracce centrali.

Contro: Seek totale leggermente maggiore rispetto a LOOK (a causa del salto di ritorno).

Il confronto tra i quattro algoritmi è riassunto nella Tabella 6.4.

Tabella 6.4: Confronto degli algoritmi di disk scheduling.

Algoritmo	Seek totale (esempio)	Starvation?	Note
FCFS	639	No	Fair, ma molto inefficiente
SSTF	238	Sì	Efficiente, ma iniquo
LOOK	210	No*	Privilegia tracce centrali
C-LOOK	325	No*	Attesa più uniforme

6.7.5 Problema comune a LOOK e C-LOOK: starvation parziale

Anche LOOK e C-LOOK hanno un problema potenziale di starvation parziale: se un certo numero di processi continua a fare richieste sullo *stesso cilindro* (o su cilindri molto vicini), il braccio della testina potrebbe non spostarsi per un periodo considerevole, lasciando in attesa le richieste lontane.

Soluzione con due code: la coda delle richieste viene suddivisa in *due sotto-code separate*:

1. Mentre il disk scheduler soddisfa le richieste della **coda A**, le nuove richieste vengono inserite nella **coda B**.
2. Quando la coda A è esaurita, le due code vengono **scambiate**: la coda B diventa la nuova coda attiva, e si apre una nuova coda B per le richieste future.

Questo garantisce che ogni richiesta venga servita entro un tempo limitato, eliminando la starvation anche in caso di richieste continue sullo stesso cilindro.

6.8 RAID

6.8.1 Motivazione e concetto

La velocità dei processori cresce seguendo la **legge di Moore** (raddoppio circa ogni 18–24 mesi), mentre la velocità dei dischi rotazionali cresce molto più lentamente (limitata da vincoli fisici meccanici). Questo **gap crescente** rappresenta un collo di bottiglia.

L'idea del RAID è sfruttare il **parallelismo**: invece di un unico disco grande e veloce (costoso), si usano più dischi “comuni” che operano in parallelo, ottenendo prestazioni e/o affidabilità superiori.

Definizione – RAID

RAID (*Redundant Array of Independent Disks*, originalmente *Inexpensive Disks*) è uno standard industriale per l'utilizzo di più dischi in parallelo. Consiste di **7 livelli** (RAID 0 – RAID 6) che rappresentano diverse architetture di distribuzione e ridondanza dei dati.

Le caratteristiche **comuni a tutti i livelli RAID**:

- Un array di dischi è visto dal sistema operativo come un **singolo disco logico virtuale**.
- I dati sono **distribuiti** (*striped*) fra i vari dischi dell'array.
- La **capacità ridondante** può essere usata per memorizzare informazioni di parità, garantendo il *recovery* in caso di guasto (ci sono quindi più copie in locale).

Nota – Considerazioni sui guasti

L'utilizzo di più dischi **aumenta** la probabilità che almeno uno di essi si guasti. Se la probabilità di guasto annuale di un singolo disco è p , con n dischi la probabilità che *almeno uno* si guasti è $1 - (1 - p)^n \approx np$ per p piccolo. RAID compensa questo con meccanismi di parità e ridondanza.

6.8.2 RAID 0 — Striping**RAID – RAID 0 – Striping**

I dati vengono distribuiti (*striped*) su più dischi in unità chiamate **strip** (settori, blocchi, o multipli di essi). Strip consecutivi si trovano su dischi diversi. **Nessuna ridondanza.**

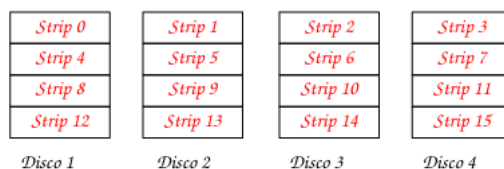


Figura 6.9: Schema di RAID 0: gli strip sono distribuiti ciclicamente sui dischi.

- **Performance:** se due richieste I/O riguardano strip su dischi diversi, possono essere servite **in parallelo**. Per grandi trasferimenti sequenziali, la velocità è circa $n \times V_r$ (dove n è il numero di dischi). Ottimo anche per molte richieste indipendenti di dimensione paragonabile allo strip.
- **Ridondanza: nessuna.** Il guasto di un qualsiasi disco causa la perdita di tutti i dati. RAID 0 tecnicamente non è un vero RAID (non c'è ridondanza), ma viene usato per applicazioni che privilegiano la velocità sull'affidabilità.

6.8.3 RAID 1 — Mirroring**RAID – RAID 1 – Mirroring**

Ogni strip viene scritto su **due dischi distinti** (uno “originale” e uno “mirror”). La capacità utile è la metà di quella totale, ma la ridondanza è massima.



Figura 6.10: Schema di RAID 1: ogni strip è duplicato su due dischi.

- **Lettura:** una richiesta può essere servita da *qualunque* dei due dischi che contengono il dato. Si può scegliere quello con il minor tempo di seek, migliorando le prestazioni in lettura.

- **Scrittura:** deve essere scritta su *tutti* i dischi che contengono la copia. Il tempo di scrittura dipende dal disco più lento (quello con seek maggiore).
- **Ridondanza:** elevatissima. In caso di guasto di un disco, i dati sono immediatamente accessibili dall'altro. Il recovery è semplice: sostituire il disco guasto e copiare il disco funzionante. Max si può rompere però un disco
- **Costo:** il costo per unità di memorizzazione raddoppia (solo metà della capacità totale è usabile per i dati).

6.8.4 RAID 4 — Parità su disco dedicato (Teorico)

RAID – RAID 4 – Parità dedicata

Usa lo striping come RAID 0, ma aggiunge un **disco di parità dedicato**. Per ogni gruppo di strip (uno per disco dati), viene calcolato uno **strip di parità** bit per bit (XOR) e memorizzato sul disco di parità.



Figura 6.11: Schema di RAID 4: il disco 5 è dedicato alla parità.

La parità si calcola con XOR bit per bit:

$$P(0-3)(i) = S_0(i) \oplus S_1(i) \oplus S_2(i) \oplus S_3(i)$$

Operazioni di lettura

- **In assenza di guasti:** si legge direttamente lo strip corrispondente dal disco appropriato.
- **In presenza di guasto:** si legge il disco di parità più tutti gli altri strip rimasti, e si ricostruisce lo strip mancante tramite XOR.

Esempio Ricostruzione RAID 4

Configurazione: 3 Dischi Dati (D_n) + 1 Disco Parità (P)

- **Dati originali:** $D_1 = 1, D_2 = 0, D_3 = 0$
- **Calcolo Parità:** $1 \oplus 0 \oplus 0 = 1 \implies P = 1$
- **Evento:** Guasto del Disco 3 (D_3)
- **Operazione di ripristino:** $D_3 = D_1 \oplus D_2 \oplus P$
- **Calcolo:** $1 \oplus 0 \oplus 1 = 0$

Risultato: Il bit originale 0 è stato recuperato correttamente.

Ottimizzazione della scrittura in RAID 4

A prima vista, scrivere un singolo strip sembra richiedere di coinvolgere *tutti* i dischi. Vediamo perché non è così.

Il problema naive. Supponiamo di voler modificare solo lo strip S_1 , sostituendolo con il nuovo valore S'_1 . Il disco di parità deve essere aggiornato di conseguenza, perché la parità si calcola su tutti gli strip del gruppo:

$$S'_4(i) = S_0(i) \oplus S'_1(i) \oplus S_2(i) \oplus S_3(i)$$

Per calcolare S'_4 con questa formula servirebbero:

- leggere S_0, S_2, S_3 (i tre strip *non* modificati),
- leggere S'_1 (già in memoria, è il nuovo valore che vogliamo scrivere),
- scrivere S'_1 sul disco 1,
- scrivere S'_4 sul disco di parità.

In totale: **3 letture + 2 scritture = 5 operazioni**. Con molti dischi diventa ancora peggio. Esiste un modo più furbo.

Il trucco: sfruttare la proprietà di XOR. XOR ha due proprietà fondamentali:

$$\begin{aligned} X \oplus X &= 0 \quad (\text{qualsiasi valore XOR sé stesso fa zero}) \\ X \oplus 0 &= X \quad (\text{qualsiasi valore XOR zero rimane invariato}) \end{aligned}$$

Ricordiamo che la *vecchia* parità vale:

$$S_4(i) = S_0(i) \oplus S_1(i) \oplus S_2(i) \oplus S_3(i)$$

Ora, nell'espressione della nuova parità, aggiungiamo e sottraiamo (ricordiamo che in XOR $-X = X$, poiché $X \oplus X = 0$) il vecchio valore S_1 :

$$S'_4(i) = S_0(i) \oplus \underbrace{S_1(i) \oplus S_1(i)}_{=0} \oplus S'_1(i) \oplus S_2(i) \oplus S_3(i)$$

Raggruppiamo in modo diverso:

$$S'_4(i) = \underbrace{S_0(i) \oplus S_1(i) \oplus S_2(i) \oplus S_3(i)}_{=S_4(i)} \oplus S_1(i) \oplus S'_1(i)$$

Quindi:

$$\boxed{S'_4(i) = S_4(i) \oplus S_1(i) \oplus S'_1(i)}$$

Nota – Cosa significa questa formula in parole

La nuova parità si ottiene prendendo la **vecchia parità** e applicando XOR con il **vecchio valore** dello strip modificato e con il **nuovo valore** dello stesso strip. Gli altri strip (S_0, S_2, S_3) non entrano più nella formula: non servono, non vengono letti.

Intuitivamente: la parità “registra” la differenza tra vecchio e nuovo. Poiché XOR è invertibile, basta sapere *cosa è cambiato* (il vecchio e il nuovo valore di S_1) per aggiornare la parità, senza rileggere tutto.

Le operazioni reali per una scrittura. Con questa formula, una scrittura richiede **solo 4 operazioni**:

1. **Leggi** S_1 dal disco 1 (il vecchio valore dello strip da modificare).
2. **Leggi** S_4 dal disco di parità (la vecchia parità).
3. **Scrivi** S'_1 sul disco 1 (il nuovo valore dello strip).
4. **Scrivi** $S'_4 = S_4 \oplus S_1 \oplus S'_1$ sul disco di parità (la nuova parità, calcolata con la formula sopra).

Rispetto alle 5 operazioni naive (3 letture + 2 scritture), ne bastano **2 letture** + **2 scritture**, indipendentemente dal numero di dischi dati nell'array. Questa ottimizzazione vale per qualsiasi dimensione del RAID.

Esempio – Verifica numerica su 4 bit

Supponiamo strip da 4 bit, con i valori correnti:

$$S_0 = 1010, \quad S_1 = 0110, \quad S_2 = 1100, \quad S_3 = 0001$$

La parità corrente è:

$$S_4 = 1010 \oplus 0110 \oplus 1100 \oplus 0001 = 0111$$

Vogliamo aggiornare S_1 con il nuovo valore $S'_1 = 1111$.

Metodo naive (ricalcolo completo):

$$S'_4 = 1010 \oplus 1111 \oplus 1100 \oplus 0001 = 1000$$

Metodo ottimizzato (formula con XOR incrementale):

$$S'_4 = S_4 \oplus S_1 \oplus S'_1 = 0111 \oplus 0110 \oplus 1111 = 1000$$

I due risultati coincidono. Con il metodo ottimizzato non abbiamo letto S_0 , S_2 , S_3 .

Il bottleneck del disco di parità. Nonostante l'ottimizzazione, rimane un problema strutturale di RAID 4: il disco di parità è coinvolto in **ogni singola scrittura** sull'intero array, anche se le scritture riguardano strip su dischi diversi.

Esempio – Scritture parallele bloccate dal disco di parità

Supponiamo che due processi vogliano scrivere contemporaneamente:

- Processo A vuole aggiornare S_1 (disco 1).
- Processo B vuole aggiornare S_6 (disco 2, gruppo diverso).

I due aggiornamenti riguardano dati su dischi fisici diversi: in linea di principio potrebbero avvenire in parallelo. Ma entrambi devono aggiornare il disco di parità (disco 5): le due scritture di parità si accodano e vengono eseguite in sequenza. Il disco di parità è il collo di bottiglia.

RAID 5 risolve questo problema distribuendo i blocchi di parità su tutti i dischi: scritture su strip diversi aggiornano dischi di parità diversi e possono procedere in parallelo.

6.8.5 RAID 5 — Parità distribuita

RAID – RAID 5 – Parità distribuita

Come RAID 4, ma i **blocchi di parità sono distribuiti** tra tutti i dischi (non su un unico disco dedicato). Questo elimina il bottleneck del disco di parità.



Figura 6.12: Schema di RAID 5: la parità è distribuita su tutti i dischi.

In RAID 5, ogni disco contiene sia strip dati sia strip di parità (relativi a strip dati su altri dischi). Le operazioni di lettura/scrittura con recovery sono identiche a RAID 4, ma le scritture parallele possono coinvolgere dischi di parità diversi, eliminando il collo di bottiglia.

- **Pro:** elimina il bottleneck del disco di parità. Stessa efficienza di spazio di RAID 4 (un disco su n è “sprecato” per la parità).
- **Contro:** in caso di guasto, la ricostruzione richiede la lettura di *tutti* gli altri dischi, cosa molto onerosa in termini di I/O. RAID 5 tollera il guasto di **un solo disco**.

6.8.6 RAID 6 — Doppia parità

RAID – RAID 6 – Doppia parità

Come RAID 5, ma si utilizzano **due strip di parità** indipendenti invece di uno, calcolati con algoritmi differenti (es. Reed-Solomon). Si perdono due dischi di capacità utile, ma si tollera il guasto di **due dischi simultaneamente**.

RAID 6 è fondamentale in sistemi con molti dischi (es. storage enterprise), dove la probabilità di un secondo guasto *durante la ricostruzione* dopo il primo guasto non è trascurabile.

6.8.7 Riepilogo dei livelli RAID

Tabella 6.5: Confronto riassuntivo dei livelli RAID principali.

Livello	Meccanismo	Capacità utile	Guasti tollerati	Uso tipico
RAID 0	Striping puro	n dischi (100%)	0	Velocità (no backup)
RAID 1	Mirroring	$n/2$ dischi (50%)	1 per coppia	Alta affidabilità
RAID 4	Parità dedicata	$n - 1$ dischi	1	Raro (bottleneck parità)
RAID 5	Parità distribuita	$n - 1$ dischi	1	Server, NAS
RAID 6	Doppia parità distrib.	$n - 2$ dischi	2	Storage enterprise

6.9 Considerazioni finali

Il sottosistema I/O è uno degli aspetti più critici delle prestazioni di un sistema operativo. Le scelte di progettazione studiate in questo capitolo — astrazione uniforme dei dispositivi, buffering/caching/spooling, disk scheduling ottimizzato, e architetture RAID — interagiscono tra loro e con i meccanismi di gestione della memoria studiati nel Capitolo 6 (page cache, memory-mapped I/O, demand paging).

I punti chiave da ricordare sono:

- Il **seek time** domina il tempo di accesso agli HDD: il disk scheduling è essenziale per le prestazioni.
- Gli **SSD** eliminano il seek, ma introducono nuove sfide (write amplification, wear leveling).
- Il **RAID** bilancia prestazioni e affidabilità attraverso il parallelismo e la ridondanza, ma non sostituisce il backup.
- **Buffering**, **caching** e **spooling** sono meccanismi distinti con scopi precisi; confonderli è un errore comune.